Протокол виконання лабораторної роботи №1

Виконав студент групи ФІ-52мн Ісаченко Нікіта.

Github: https://github.com/misach2135/distributed_systems_labs/pull/1

Мета: реалізувати частину банківської системи на основі трьох мікросервісів, взаємодія між якими відбувається на основі HTTP-протоколу, на основі парадигми REST API.

Хід роботи

Мікросервіси були реалізовані мовою Rust за допомогою фреймворка axum, асинхронного середовища виконання tokio та HTTP-клієнта reqwest. Проєкт організований як Cargo workspace, що складається з трьох сервісів та спільного крейту `common`.

Система складається з трьох незалежних мікросервісів:

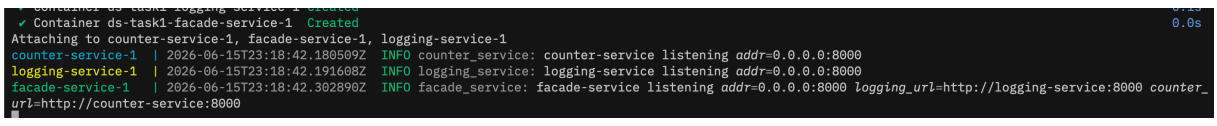
- **facade-service** — єдина точка входу для клієнта. Приймає транзакцію, генерує для неї унікальний `transaction_id` (UUID v4) та часову мітку (timestamp), після чого **паралельно** (за допомогою `tokio::join!`) надсилає її до logging-service та counter-service і повертає клієнту відповідь.
- **logging-service** — зберігає всі отримані транзакції у пам'яті у вигляді хеш-таблиці з ключем `transaction_id` та повертає їх за запитом.
- **counter-service** — обчислює та зберігає у пам'яті підсумковий баланс для кожного клієнта (ключ — `user_id`), застосовуючи операції поповнення/дебетування.

Спільний крейт `common` містить єдиний протокол обміну (структури транзакцій, відповідей та балансів із похідними `Serialize`/`Deserialize`), завдяки чому всі сервіси гарантовано використовують однаковий формат повідомлень. Між сервісами дані передаються у форматі JSON.

Для потокобезпечного зберігання стану в пам'яті використовується `DashMap` — хеш-таблиця з блокуванням на рівні окремого ключа. Це дозволяє counter-service атомарно виконувати операцію «прочитати-змінити-записати» над балансом і коректно обробляти одночасні запити навіть до одного й того самого рахунку.

Для зручності, були також додані контейнер-файли для запуску мікросервісів у контейнерах. Для запуску усіх необхідних контейнерів зручно використовувати `docker-compose`. У `docker-compose.yaml` усі сервіси розміщені в одній bridge-мережі, тому вони звертаються один до одного за іменем (наприклад, `http://logging-service:8000`).

```
docker compose up --build
```

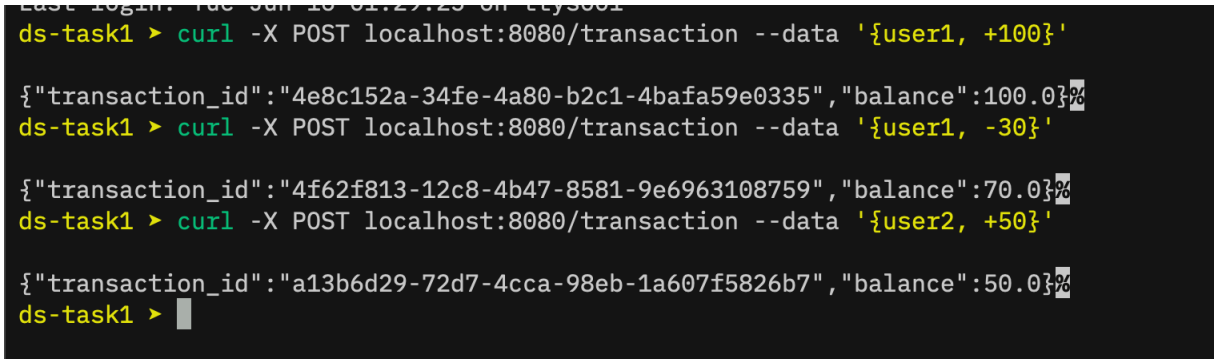


Користувач комунікує з мікросервісами через фасад. Доступні ендпоїнти:

```
pub fn router(downstream: Downstream) -> Router {
    Router::new()
```

```
        .route("/transaction", post(transaction))
        .route("/user/{user_id}", get(user))
        .route("/accounts", get(accounts))
        .route("/metrics", get(metrics))
        .route("/metrics/reset", post(reset_metrics))
        .route("/health", get(health))
        .with_state(downstream)
}
```

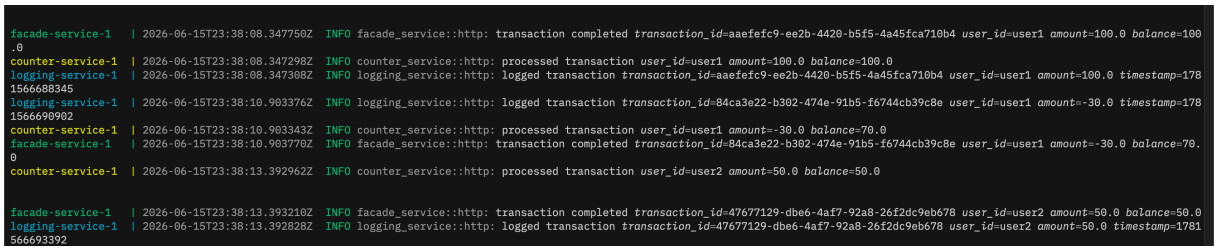
Для додавання повідомлення використовується ендопоїнт `/transaction`. Клієнтське повідомлення передається у власному (не-JSON) форматі `{user_Id, +/- amount}`, який розбирається за допомогою реалізацій трейтів `FromStr`/`Display` та регулярного виразу у крейті `common`. Отримавши повідомлення, facade-service доповнює його `transaction_id` і timestamp та оркеструє виклики до інших сервісів. Взаємодія з цим ендпоїнтом через curl виглядає таким чином:



Для того, щоб вивести інформацію про користувача(його баланс та транзакції), можна використати ендпоїнт `/user/{user_id}`. Сам користувач, точніше запис про користувача, створюється коли для відповідного `user_id` виконується певна транзакція.



При цьому, для кожного мікросервісу виводяться логи:



Окремо facade-service вимірює, скільки часу займає виклик до кожного з сервісів. Тривалість кожного віддаленого виклику заміряється через `tokio::time::Instant` і накопичується в атомарних лічильниках (`AtomicU64`) окремо для logging-service та counter-service. Накопичені значення доступні через ендпоїнт `/metrics`, а `/metrics/reset` дозволяє скинути їх перед черговим запуском тесту.

Також у лабораторній роботі реалізований стрес-тест за двома сценаріями, які вимагались у методичних вказівках до лабораторної роботи.

- Сценарій А: 10 клієнтів одночасно роблять по 10К однакових транзакцій по додаванню 1 на свій рахунок. У результаті кінцеве значення балансу на 10-и рахунках має бути по 10К.
- Сценарій Б: 10 клієнтів одночасно роблять по 10К однакових транзакцій по додаванню 1 на один і той самий рахунок. У результаті кінцеве значення балансу на одному рахунках має бути 100К.

Інтеграційний стрес-тест написаний мовою python, і розміщений у папці tests. Результати інтеграційних тестів за двома сценаріями у Table 1.

Test scenarios	**Task 1 (in-mem)**	**Task 3 (DB)**	**Task 5 (final)**
10 accounts	Total time: **36.327 s**	Total time:	Total time:
	logging-service contribution: **224643.4 ms**	logging-service contribution:	logging-service contribution:
	counter-service contribution: **37546.5 ms**	counter-service contribution:	counter-service contribution:
1 account	Total time: **47.917 s**	Total time:	Total time:
	logging-service contribution: **148583.7 ms**	logging-service contribution:	logging-service contribution:
	counter-service contribution: **72849.0 ms**	counter-service contribution:	counter-service contribution:

Table 1: Таблиця порівняння результатів

Висновки:

Різниця у швидкості між сценаріями пояснюється механізмами синхронізації паралельного доступу до пам'яті: падіння пропускної здатності у тесті зі спільним рахунком (2086 запитів/с) порівняно з окремими рахунками (2752 запитів/с) зумовлене чергою потоків за доступ до єдиного ключа у хеш-таблиці counter-service (lock contention), тоді як оновлення різних

рахунків відбувається без взаємних блокувань. Разом з тим стабільно вищі витрати часу на logging-service є абсолютно закономірними з огляду на поставлене завдання, оскільки аллокація пам’яті для збереження повної структури кожної транзакції та синхронні I/O операції виводу в консоль потребують значно більше ресурсів, ніж елементарна математична зміна числа в балансі, що загалом доводить правильність реалізації системи та її здатність гарантувати консистентність даних в умовах високої конкурентності.